

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

CASE STUDY:

Code of Conduct:

I **ANISH K/192521288** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1 A multinational software company develops a large ERP application consisting of 120 dynamically linked libraries. During execution, users complain that startup time is high while memory utilization remains low. As the system architect, analyze whether static linking or dynamic linking is the better choice. Recommend an optimized loading strategy by considering compile time, load time, execution time, and shared libraries. Justify your decision with suitable memory diagrams.

2 A cloud server simultaneously executes AI applications, database services, and web applications. Although 128 GB RAM is installed, memory allocation failures occur frequently. Investigate the causes by analyzing dynamic storage management, external fragmentation, and internal fragmentation. Design a memory management strategy that minimizes memory wastage while maintaining high throughput.

3 An airline reservation system experiences severe delays whenever thousands of customers book tickets simultaneously. The operating system frequently swaps processes between RAM and disk. Analyze how swapping affects CPU utilization and response time. Propose modifications to reduce swapping overhead without upgrading hardware.

4 A hospital management system allocates memory using contiguous memory allocation. After months of continuous operation, several large applications fail to load despite sufficient free memory. Investigate the reason using fragmentation concepts. Compare First Fit, Best Fit, Worst Fit, and Next Fit strategies, and recommend the most appropriate allocation technique for this scenario.

5 A banking application initially uses contiguous memory allocation but is later migrated to paging. Compare the performance of both approaches under heavy transaction loads. Evaluate page table overhead, fragmentation, memory utilization, and execution efficiency before recommending the migration strategy.

6 A gaming company develops an open-world game with multiple independent modules such as graphics, AI, physics engine, and audio processing. Analyze how segmentation can improve memory organization compared to paging. Design an appropriate segment table and explain how logical addresses are translated into physical addresses.

7 A software development company uses both segmentation and paging in its operating system. During debugging, developers notice increased address translation time. Analyze the advantages and disadvantages of segmentation with paging compared to

pure paging and pure segmentation. Recommend an optimized architecture for the organization.

8 A mobile operating system allows users to run multiple applications simultaneously, exceeding the available physical memory. Analyze how virtual memory enables efficient multitasking. Explain the consequences if virtual memory is disabled, and propose improvements for better memory performance.

9 An online video editing application loads only required portions of videos into memory. Users report occasional delays when editing large files. Analyze how demand paging influences system performance. Suggest methods to reduce page faults without increasing RAM capacity.

10 A data analytics server experiences frequent page faults while processing large datasets. After monitoring memory references, the administrator notices repetitive loading and replacement of the same pages. Analyze the effectiveness of FIFO, LRU, Optimal, and Clock page replacement algorithms. Recommend the most suitable algorithm for this workload with proper justification.

11 A university examination server crashes during online examinations due to excessive page faults. Investigate whether thrashing is responsible for the failure. Analyze CPU utilization, page fault frequency, and working set size. Design an improved memory allocation policy to eliminate thrashing.

12 A cloud provider hosts hundreds of virtual machines on a single physical server. During peak hours, several VMs become unresponsive. Analyze the role of working sets in preventing thrashing. Propose a memory allocation framework that dynamically adjusts working set sizes based on workload patterns.

13 A multimedia company stores high-definition video files on HDDs while frequently accessed metadata is stored on SSDs. Analyze how disk devices influence virtual memory performance, swapping speed, and page replacement efficiency. Recommend a storage hierarchy that minimizes access latency.

14 A software company plans to redesign its operating system memory manager. The proposed system integrates dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping. Evaluate whether this integrated approach

improves overall system performance. Identify possible bottlenecks and propose architectural improvements.

15 An e-commerce platform experiences inconsistent response times during festival sales. System logs reveal excessive dynamic library loading, frequent page replacements, and swapping activity. Analyze the interaction among dynamic linking, demand paging, and swapping. Develop a comprehensive optimization strategy that improves performance while minimizing memory consumption.

16 A research laboratory develops a scientific simulation requiring large matrices exceeding available RAM. Compare the effectiveness of virtual memory, segmentation, paging, and segmentation with paging. Recommend the most efficient memory management model by considering scalability, performance, and memory protection.

17 A cybersecurity organization requires strict memory protection between user processes and kernel processes. Analyze how segmentation, paging, and segmentation with paging provide isolation and protection. Design a secure memory architecture capable of preventing unauthorized memory access while maintaining execution efficiency.

18 A fintech company is designing a next-generation operating system for financial servers. The system must support secure memory allocation, efficient page replacement, fast loading of shared libraries, and minimal disk I/O. Develop a complete memory management framework integrating dynamic storage management, paging, virtual memory, working sets, and SSD storage. Justify each design decision.

19 An autonomous vehicle runs perception, navigation, and object detection modules concurrently on limited embedded memory. Analyze how demand paging, page replacement policies, virtual memory, and working set models influence real-time performance. Recommend a customized memory management strategy that guarantees deterministic execution with minimal latency.

20 A technology company plans to build an operating system for future AI-powered supercomputers. The system must efficiently support millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. Design a complete memory management architecture integrating linkers, dynamic linking, dynamic storage management, swapping, contiguous allocation, paging, segmentation, segmentation with paging, virtual memory, demand paging,

page replacement, thrashing prevention, working sets, and advanced disk management. Evaluate trade-offs among performance, scalability, reliability, and cost.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Question 1

Case Analysis

The ERP application contains 120 dynamically linked libraries. Users experience high startup time even though memory usage is low. The objective is to determine whether static linking or dynamic linking is more suitable and recommend an optimized loading strategy.

Application of OS Concepts

Static Linking

Dynamic Linking

Compile-time Linking

Load-time Linking

Run-time Dynamic Loading

Shared Libraries

Analysis

In static linking, all required libraries are included in the executable during compilation. This increases executable size and duplicates library code in memory for every process.

In dynamic linking, libraries are loaded only when required and can be shared among multiple processes. Although startup time may slightly increase due to library loading, memory utilization is significantly improved.

Since the ERP system contains many common libraries, dynamic linking allows multiple users to share the same library in memory instead of creating separate copies.

Recommended Solution

Continue using dynamic linking.

Use lazy loading (on-demand loading) so libraries are loaded only when first accessed.

Preload frequently used libraries during system startup.

Cache commonly used shared libraries in memory.

Memory Diagram

Static Linking

Program

```
+-----+
| Code + All Libraries |
+-----+
```

Each process stores all libraries separately.

Dynamic Linking

Program

```
+-----+
| Code  |
+-----+
|
V
```

Shared Library

```
+-----+
| Shared DLLs |
+-----+
```

Many processes share the same library.

Conclusion:

Dynamic linking with lazy loading provides better memory utilization, easier updates, and improved scalability. Preloading frequently used libraries reduces startup delay while maintaining efficient memory usage.

Question 2

Case Analysis

A cloud server with 128 GB RAM still experiences memory allocation failures while running AI applications, databases, and web services simultaneously.

Application of Concepts

Dynamic Storage Management

External Fragmentation

Internal Fragmentation

Memory Allocation

Analysis

Memory failures occur because free memory is fragmented.

External Fragmentation: Free memory exists but is scattered into small blocks.

Internal Fragmentation: Allocated memory blocks are larger than required, causing wasted space.

Large AI applications require continuous memory blocks, making allocation difficult even when sufficient total memory exists.

Proposed Solution

Implement paging to eliminate external fragmentation.

Use dynamic memory allocation instead of fixed-size allocation.

Apply memory compaction periodically.

Separate memory pools for AI, databases, and web services.

Monitor memory utilization continuously.

Memory Illustration

Before

|Used|Free|Used|Free|Used|Free|

Large block unavailable.

After Paging

|Page1|Page2|Page3|Page4|

Pages can be allocated anywhere.

Conclusion:

Paging combined with dynamic storage management minimizes fragmentation, improves memory utilization, and increases overall throughput.

Question 3

Case Analysis

Thousands of airline customers book tickets simultaneously, causing excessive swapping between RAM and disk, resulting in poor response time.

Application of Concepts

Swapping

CPU Utilization

Response Time

Multiprogramming

Analysis

Swapping moves processes between RAM and secondary storage.

When excessive swapping occurs:

CPU waits for disk operations.

Disk I/O increases.

Response time becomes longer.

Overall throughput decreases.

Frequent swapping creates a bottleneck because disk access is much slower than RAM access.

Proposed Solution

Reduce the degree of multiprogramming.

Implement demand paging.

Increase process scheduling efficiency.

Keep frequently used processes in memory.

Use SSD storage to reduce swapping time.

Swapping Diagram

RAM

+-----+

|Process A|

|Process B|

+-----+

|

Swapping

|

Disk

+-----+

|Process C|

|Process D|

+-----+

Conclusion:

Demand paging and efficient scheduling significantly reduce swapping overhead, improving CPU utilization and response time without requiring additional hardware.

Question 4

Case Analysis

A hospital management system uses contiguous memory allocation. Large applications fail to load despite sufficient free memory.

Application of Concepts

External Fragmentation

First Fit

Best Fit

Worst Fit

Next Fit

Analysis

The problem occurs because free memory is divided into many small blocks that cannot satisfy large requests.

Comparison

Strategy	Advantages	Disadvantages
First Fit	Fast allocation	External fragmentation
Best Fit	Less memory waste	Creates many small holes
Worst Fit	Large free blocks remain	Poor memory utilization
Next Fit	Faster than First Fit	Higher fragmentation

Recommended Solution

Use Best Fit with periodic memory compaction or replace contiguous allocation with paging, which removes external fragmentation completely.

Fragmentation Diagram

Memory

|Used|Free|Used|Free|Used|Free|

Total free memory = Large

Continuous block = Not Available

Conclusion:

Paging is the best long-term solution because it eliminates external fragmentation and improves memory allocation efficiency.

Question 5

Case Analysis

A banking system migrates from contiguous memory allocation to paging. The performance of both methods must be evaluated.

Application of Concepts

Paging

Contiguous Allocation

Page Table

Fragmentation

Memory Utilization

Analysis

Contiguous Allocation

Advantages:

Simple implementation.

Fast address translation.

Disadvantages:

External fragmentation.

Difficult memory expansion.

Poor utilization under heavy load.

Paging

Advantages:

Eliminates external fragmentation.

Better memory utilization.

Supports virtual memory.

Efficient for multitasking.

Disadvantages:

Requires page tables.

Small internal fragmentation.

Slight address translation overhead.

Comparison

Feature	Contiguous	Paging
External Fragmentation	High	None
Internal Fragmentation	Low	Small
Memory Utilization	Medium	High
Page Table Overhead	None	Present
Multitasking	Poor	Excellent
Scalability	Low	High

Recommendation

The banking application should migrate to paging because transaction processing requires high scalability, better memory utilization, and support for multiple concurrent users.

Conclusion:

Although paging introduces page table overhead, its advantages in memory management, multitasking, and performance make it the preferred choice for modern banking systems.

Question 6

Case Analysis

A gaming company develops an open-world game with independent modules such as graphics, AI, physics engine, and audio processing. The task is to analyze how segmentation improves memory organization compared to paging and design a suitable segment table.

Application of OS Concepts

Segmentation

Logical Address

Physical Address

Segment Table

Memory Protection

Analysis

Segmentation divides a program into logical units such as code, data, stack, graphics, AI, and audio. Each segment has its own base address and limit.

Unlike paging, segmentation reflects the program structure, making memory management easier. Each module can be loaded, modified, or protected independently.

Sample Segment Table

Segment	Base Address	Limit
Graphics	1000	500
AI Engine	2000	400
Physics	3000	350
Audio	4000	250

Logical to Physical Address Translation

Physical Address = Base Address + Offset

Example:

Graphics Segment

Base = 1000

Offset = 150

Physical Address = 1150

Diagram

Logical Address

(Segment No., Offset)

|

V

Segment Table

(Base, Limit)

|

V

Physical Memory

Conclusion:

Segmentation is more suitable than paging because it organizes memory according to program modules, simplifies debugging, and provides better memory protection.

Question 7

Case Analysis

A software company uses segmentation with paging, but developers notice increased address translation time.

Application of Concepts

Paging

Segmentation

Segmentation with Paging

Address Translation

Analysis

Comparison

Technique	Advantages	Disadvantages
Paging No	external fragmentation	Internal fragmentation
Segmentation Logical	organization	External fragmentation
Segmentation with Paging	Logical structure	Two-level address translation

Segmentation with paging combines the benefits of both methods but increases translation time because:

Segment table lookup

Page table lookup

Memory access

are required before accessing data.

Recommended Solution

Use Translation Lookaside Buffer (TLB) to reduce lookup time.

Keep frequently accessed pages in TLB.

Optimize page size.

Use hierarchical page tables.

Conclusion:

Segmentation with paging provides excellent memory organization and protection. Using TLB minimizes translation overhead while maintaining high performance.

Question 8

Case Analysis

A mobile operating system runs many applications simultaneously, exceeding available physical memory.

Application of Concepts

Virtual Memory

Demand Paging

Multitasking

Page Fault

Analysis

Virtual memory allows applications to use more memory than the available RAM by storing inactive pages on disk.

Benefits

- Supports multiple applications.
- Improves multitasking.
- Efficient memory utilization.
- Allows execution of large applications.
- If Virtual Memory is Disabled
- Large applications cannot run.
- Fewer applications execute simultaneously.
- Frequent application crashes.
- Poor multitasking performance.
- Suggested Improvements
- Use SSD for faster swapping.
- Apply demand paging.
- Use LRU page replacement.
- Increase TLB efficiency.

Diagram

Applications

|

Virtual Memory

|

RAM <-----> SSD

Conclusion:

Virtual memory is essential for modern mobile operating systems because it enables efficient multitasking while improving memory utilization.

Question 9

Case Analysis

An online video editor loads only required portions of videos into memory. Users experience delays while editing large files.

Application of Concepts

Demand Paging

Page Fault

Locality of Reference

Page Replacement

Analysis

Demand paging loads pages only when required.

Advantages

- Lower RAM usage.
- Faster application startup.
- Efficient memory utilization.
- Disadvantages
- Frequent page faults increase delays.
- Large video files require repeated disk access.
- Methods to Reduce Page Faults
- Use LRU page replacement.
- Increase page cache.
- Apply read-ahead (prefetching).
- Store temporary files on SSD.
- Keep frequently accessed pages in memory.

Diagram

Video File

|

Demand Paging

|

RAM

|

Page Fault

|

Load Missing Page

Conclusion:

Demand paging improves memory efficiency, but combining it with LRU and SSD storage significantly reduces editing delays.

Question 10

Case Analysis

A data analytics server repeatedly loads and replaces the same pages while processing large datasets.

Application of Concepts

FIFO

LRU

Optimal

Clock Algorithm

Page Replacement

Analysis

Comparison

Algorithm	Advantages	Disadvantages
FIFO	Very simple	May replace important pages
LRU	Uses recent access history	Higher implementation cost
Optimal	Lowest page faults	Not practical in real systems
Clock	Efficient approximation of LRU	Slightly less accurate

Since the workload repeatedly accesses recently used pages, FIFO performs poorly because it removes pages regardless of usage.

LRU keeps recently used pages in memory, reducing repeated page faults.

Recommended Algorithm

LRU (Least Recently Used)

Reasons:

- Lower page fault rate.
- Better performance for repeated access.
- Suitable for analytics workloads.
- Improves CPU utilization.

Diagram

Memory Frames

Page A

Page B

Page C

↓

New Page Arrives

LRU removes

Least Recently Used Page

Conclusion:

LRU is the most suitable page replacement algorithm for large data analytics because it minimizes repeated page faults and improves execution efficiency.

Question 11

Case Analysis

A university examination server crashes during online exams due to excessive page faults. The task is to determine whether thrashing is responsible and design a policy to eliminate it.

Application of OS Concepts

Thrashing

Page Fault Frequency (PFF)

Working Set

CPU Utilization

Virtual Memory

Analysis

Thrashing occurs when the operating system spends more time swapping pages than executing processes.

Symptoms

Very high page fault rate.

Low CPU utilization.

Continuous disk activity.

Slow system response.

The examination server experiences these symptoms because many student processes compete for limited memory.

Proposed Solution

Implement the Working Set Model.

Monitor Page Fault Frequency (PFF).

Reduce the degree of multiprogramming.

Allocate sufficient frames to active processes.

Use LRU page replacement.

Diagram

Many Page Faults



Disk Swapping



CPU Idle



Thrashing

Conclusion:

Using the Working Set Model and controlling page fault frequency prevents thrashing and improves server performance during online examinations.

Question 12

Case Analysis

A cloud provider hosts hundreds of virtual machines (VMs) on one server. During peak hours, several VMs become unresponsive due to memory pressure.

Application of OS Concepts

Working Set

Thrashing

Dynamic Memory Allocation

Virtual Machines

Analysis

Each virtual machine requires a minimum number of memory pages (working set). If insufficient memory is allocated, frequent page faults occur, causing thrashing.

Allocating memory equally to every VM is inefficient because workloads change dynamically.

Proposed Memory Allocation Framework

Continuously monitor each VM's working set.

Increase memory for heavily loaded VMs.

Reduce memory for idle VMs.

Use dynamic memory balancing.

Use SSD-based swapping for inactive pages.

Framework Diagram

Virtual Machines

|

Working Set Monitor

|

Dynamic Memory Allocation

|

RAM / SSD

Conclusion:

Dynamic working-set allocation improves memory utilization, prevents thrashing, and ensures better cloud server performance.

Question 13

Case Analysis

A multimedia company stores HD video files on HDDs while frequently accessed metadata is stored on SSDs. The objective is to analyze how storage devices affect virtual memory performance.

Application of OS Concepts

Virtual Memory

Swapping

HDD

SSD

Page Replacement

Analysis

HDD

High storage capacity.

Slow access time.

Slower swapping operations.

SSD

Very fast access speed.

Low latency.

Faster page replacement and swapping.

Using SSD for frequently accessed data significantly improves virtual memory performance.

Recommended Storage Hierarchy

Storage	Purpose
RAM	Active pages
SSD	Swap space, metadata
HDD	Large video files, backups

Diagram

CPU

|

RAM

|

SSD (Swap)

|

HDD (Storage)

Conclusion:

Using SSD for swapping and metadata while storing large files on HDD minimizes access latency and improves overall system performance.

Question 14

Case Analysis

A software company plans to redesign its operating system memory manager by integrating dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping.

Application of OS Concepts

Dynamic Linking

Demand Paging

Segmentation

Virtual Memory

SSD Swapping

Analysis

- Each technique contributes differently:
- Dynamic Linking reduces executable size.
- Demand Paging loads only required pages.
- Segmentation improves logical organization.
- Virtual Memory supports large applications.
- SSD swapping speeds up page transfer.
- Possible Bottlenecks
- Excessive page faults.
- Address translation overhead.
- SSD wear due to continuous swapping.
- High memory management complexity.
- Architectural Improvements
- Use Translation Lookaside Buffer (TLB).
- Apply LRU page replacement.
- Implement Working Set Model.
- Use predictive prefetching.
- Compress inactive memory pages.
- Architecture Diagram

Diagram

Application

|

Dynamic Linking

|

Segmentation

|

Demand Paging

|

Virtual Memory

|

SSD Swap

Conclusion

Integrating these memory management techniques provides higher performance, better scalability, and efficient memory utilization when supported by TLB and working-set optimization.

Question 15

Case Analysis

An e-commerce platform experiences inconsistent response times during festival sales due to excessive dynamic library loading, page replacement, and swapping.

Application of OS Concepts

Dynamic Linking

Demand Paging

Swapping

Page Replacement

Memory Optimization

Analysis

During peak traffic:

- Dynamic libraries increase startup overhead.
- Demand paging generates more page faults.
- Swapping increases disk activity.
- Excessive page replacement reduces performance.

All these factors together increase response time:

- Comprehensive Optimization Strategy
- Preload frequently used shared libraries.
- Use demand paging with intelligent prefetching.
- Implement LRU page replacement.
- Reduce swapping by increasing working-set allocation.
- Store swap space on SSD.
- Monitor memory utilization continuously.

Optimization Flow

Shared Libraries

|

Dynamic Linking

|

Demand Paging

|

LRU Replacement

|

SSD Swapping

|

Fast Response Time

Conclusion:

Combining optimized dynamic linking, efficient paging, intelligent page replacement, and SSD-based swapping minimizes memory consumption while maintaining fast response times during heavy workloads.

Question 16

Case Analysis

A research laboratory develops scientific simulations using very large matrices that exceed the available RAM. The task is to compare virtual memory, segmentation, paging, and segmentation with paging and recommend the best memory management model.

Application of OS Concepts

Virtual Memory

Paging

Segmentation

Segmentation with Paging

Memory Protection

Analysis

Technique	Advantages	Disadvantages
Virtual Memory	Executes programs larger than RAM	Higher page faults
Paging Eliminates	external fragmentation	Internal fragmentation
Segmentation	Logical memory organization	External fragmentation
Segmentation with Paging	Combines protection	Complex implementation

Scientific simulations require large memory, high scalability, and secure memory access. Segmentation with paging provides logical organization while paging efficiently manages physical memory.

Recommended Solution

- Use Segmentation with Paging.
- Implement Virtual Memory for large datasets.
- Use LRU page replacement.
- Store swap space on SSD.

Diagram

Application

|

Segmentation

|

Paging

|

Virtual Memory

|

Physical Memory

Conclusion:

Segmentation with paging combined with virtual memory provides the best scalability, protection, and memory utilization for scientific simulations.

Question 17

Case Analysis

A cybersecurity organization requires strong isolation between user processes and the kernel to prevent unauthorized memory access.

Application of OS Concepts

Segmentation

Paging

Segmentation with Paging

Memory Protection

Analysis

Comparison

Technique	Protection
Segmentation	Protects logical program sections
Paging	Protects individual pages using access bits
Segmentation + Paging	Highest protection and flexibility

Segmentation separates user and kernel memory logically, while paging enforces access permissions at the page level.

Secure Memory Architecture

- Separate kernel and user address spaces.
- Use read/write/execute protection bits.
- Enable paging with privilege levels.
- Maintain separate page tables for kernel and user processes.
- Validate every memory access through the MMU.

Diagram

CPU

|

MMU

|

+-----+

| Kernel |

+-----+

| User Apps |

+-----+

Conclusion:

Segmentation with paging offers the highest level of memory protection, ensuring secure execution while maintaining efficient memory management.

Question 18

Case Analysis

A fintech company is designing an operating system for financial servers that requires secure memory allocation, efficient page replacement, fast shared library loading, and minimal disk I/O.

Application of OS Concepts

Dynamic Storage Management

Paging

Virtual Memory

Working Set

SSD Storage

Dynamic Linking

Analysis

The operating system should integrate multiple memory management techniques:

- Dynamic Linking to share common libraries and reduce memory usage.
- Paging to eliminate external fragmentation.
- Virtual Memory to support large applications.
- Working Set Model to avoid thrashing.
- SSD Storage for faster swapping.
- Proposed Framework
- Dynamic memory allocation.
- Dynamic linking for shared libraries.
- Demand paging with LRU replacement.
- Working-set monitoring.
- SSD-based swap area.
- Continuous memory performance monitoring.

Framework Diagram

Applications

|

Dynamic Linking

|

Demand Paging

|

Working Set

|

SSD Swap

|

RAM

Conclusion:

An integrated memory management framework improves security, reduces disk I/O, increases performance, and supports reliable financial transactions.

Question 19

Case Analysis

An autonomous vehicle simultaneously runs perception, navigation, and object detection modules using limited embedded memory. The system requires deterministic real-time performance.

Application of OS Concepts

Demand Paging

Page Replacement

Virtual Memory

Working Set

Real-Time Systems

Analysis

Real-time systems cannot tolerate unpredictable delays caused by excessive page faults.

Demand paging saves memory but may introduce latency. A suitable page replacement policy and working-set allocation are required to ensure timely execution.

Recommended Memory Strategy

- Keep critical pages permanently in RAM.
- Allocate fixed working sets for real-time tasks.
- Use LRU for non-critical pages.
- Minimize swapping.
- Use SSD only for non-critical background operations.

Diagram

Sensors

|

Perception

Navigation

Object Detection

|

RAM (Critical Pages)

|

SSD (Background Data)

Conclusion:

Maintaining critical pages in RAM while applying working-set management ensures deterministic execution and minimal latency for autonomous vehicles.

Question 20

Case Analysis

A technology company plans to develop an operating system for future AI-powered supercomputers supporting millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage.

Application of OS Concepts

- Linkers
- Dynamic Linking
- Dynamic Storage Management
- Swapping
- Contiguous Allocation
- Paging
- Segmentation
- Segmentation with Paging
- Virtual Memory
- Demand Paging
- Page Replacement
- Thrashing Prevention
- Working Set
- Disk Management

Analysis

The operating system must integrate advanced memory management techniques to maximize performance and scalability.

- Dynamic linking reduces executable size.
- Paging eliminates external fragmentation.
- Segmentation organizes memory logically.
- Virtual memory enables execution of large AI workloads.
- Demand paging loads only required pages.
- LRU reduces unnecessary page faults.
- Working-set monitoring prevents thrashing.
- SSDs accelerate swapping.
- Distributed storage manages massive datasets efficiently.
- Proposed Memory Architecture
- Dynamic linking for shared libraries.
- Paging with segmentation.
- Virtual memory with demand paging.
- LRU page replacement.
- Working-set monitoring.
- SSD-based swapping.
- Distributed storage for AI datasets.
- Continuous memory optimization using system monitoring.

Architecture Diagram

Applications

|

Dynamic Linking

|

Segmentation

|

Paging

|

Virtual Memory

|

LRU Replacement

|
Working Set Monitor

|
SSD Swap

|
Distributed Storage

Trade-off Analysis

Factor	Result
Performance	Very High
Scalability	Excellent
Reliability	High
Memory Utilization	Excellent
Cost	Higher due to SSD and advanced hardware

Conclusion:

An integrated architecture combining dynamic linking, paging, segmentation, virtual memory, working-set management, LRU page replacement, SSD swapping, and distributed storage provides excellent performance, scalability, reliability, and efficient memory utilization for future AI-powered supercomputers.